

Linuc 教科书

李思怡 | 今天修改

第1章Linuxのインストールと仮想マシン・コンテナの利用

◆1.1 Linuxのインストール、起動、接続、切断と停止

1.1.1 DVD-ROMからのLinuxインストール

- 核心要点：
 - 安装 Linux 需要从 **安装介质 (DVD、ISO)** 启动
 - 电脑默认会从硬盘启动，所以需要去 **BIOS/UEFI 设置启动顺序**(**BIOS/UEFIで設定できるものは起動デバイスの優先順位, デバイスの有効/無効化**)，把 DVD/USB 放到硬盘之前
 - 开机时按 **F2 / Delete** 等进入 BIOS/UEFI (不同主板按键不同，但通常屏幕会提示)
 - 启动后，会进入 **安装菜单**，常见选项有：
 - 安装 Linux
 - 内存测试
 - 系统修复
 - 在安装过程中，需要逐步设置：
 - i. 时区 (Time Zone)
 - ii. 语言/Locale
 - iii. 键盘布局
 - iv. 主机名 (Hostname)
 - v. 用户名、密码
 - vi. 磁盘分区 (自动 or 手动)
 - vii. 需要安装的软件组 (最小系统、Web服务器、DB服务器等)

👉 **通俗理解**：这就像买了一台新电脑，要先告诉它「从光盘启动」，接着一步一步填表格（时区、用户名、密码），再选硬盘怎么分。

🌟 BIOS/UEFI 能做的事

这是**最底层固件设置界面**，能配置：

- 系统时间和日期
- 电源管理
- **启动顺序 (从硬盘、USB、网络启动...)**
- **某些硬件设备开/关 (比如网卡、声卡、USB 控制器)**

1.1.2 Linuxの起動と停止

- **启动**：安装完成 → 系统重启 → 出现 `login:` 提示符
 - 输入用户和密码就能登录
- **关机/重启命令**：
 - `sudo shutdown -h now` # 立刻关机
 - `sudo shutdown -r now` # 立刻重启
 - `reboot` # 重启 (等价于上面)
 - `halt` # 停机 (关机)
 - `poweroff` # 关机
- 普通用户要用 **sudo**，root 用户可以直接用命令

👉 通俗理解：

就像开关机按钮，Linux 用命令行完成。`-h` = halt（停机），`-r` = reboot（重启）。

Linux 里几个“关机命令”到底有什么关系

◆ 1. 普通用户和 root 的区别

- **普通用户**：权限不够，不能直接关机或重启 → 需要 `sudo` 提升权限。
- **root 用户**：是超级管理员，可以直接执行这些命令。

所以，区别只是**谁能用，不是命令本身不同**。

◆ 2. 各个命令的关系

Linux 历史里，有很多命令都能实现关机/重启，但本质上很多都是**调用同一套系统接口**。

1. shutdown

- `shutdown -h now` → halt（关机，停机）
- `shutdown -r now` → reboot（重启）
👉 这是最标准、最“官方”的用法。

2. reboot

- 其实就是调用 `shutdown -r now`。
👉 简化版写法。

3. halt

- 本义是“停机”。
- 在现代 Linux 里，通常等价于 `poweroff`，有时只是停止 CPU，不一定断电（取决于系统）。

4. poweroff

- 直接断电关机。
 - 在很多系统里就是调用 `halt`。
-

◆ 3. 你理解的等价关系

完全正确 ✅：

- `sudo shutdown -h now`
≈ `sudo poweroff`
≈ `sudo halt`
- `sudo shutdown -r now`
≈ `sudo reboot`

（前提是现代 Linux 系统，像 systemd 之后，确实都统一到同一个后台调用）

◆ 4. 记忆法

可以这么记：

- **shutdown** → “最全能”，带参数 `-h` 停机 / `-r` 重启
- **reboot** → 快捷方式，等于 `shutdown -r now`
- **poweroff / halt** → 快捷方式，等于 `shutdown -h now`

👉 平时考试背口诀：

```
-h = halt（关机），-r = reboot（重启）
reboot = shutdown -r now
poweroff = shutdown -h now
```

如果你已经是 **root 用户**，那么：

- `halt` → 直接停机（现代 Linux 下通常就是关机）
- `poweroff` → 直接关机（和 `halt` 几乎一样）
- `reboot` → 直接重启

👉 root 用户不需要 `sudo`，因为已经有最高权限。

🔥 但是有两个小补充点要知道（考试也可能考）：

1. 传统区别：

- `halt` → 只停 CPU（不一定断电）
 - `poweroff` → 真正断电
- 不过现在 `systemd` 统一后，几乎都等价了。

2. shutdown 更正规：

- 在多用户环境下，`shutdown` 会提前通知所有登录用户“系统即将关机”，给他们时间保存工作。
- `halt/poweroff/reboot` 就是直接执行，不通知。

✅ 所以，如果你是 root：

- 想正规、安全：`shutdown -h now`
- 想快点、自己一台机：`halt` / `poweroff`

1.1.3 SSHを使ったりリモートログイン

- **为什么要 SSH：**不用跑到服务器机房插显示器，可以远程管理
- **关键文件**（在 `~/.ssh` 下）：
 - `known_hosts`：保存你连过的服务器信息
 - `id_rsa`：你的私钥
 - `id_rsa.pub`：你的公钥（要复制到服务器上）
- **配置公钥免密登录：**
- `ssh-copy-id -i ~/.ssh/id_rsa.pub linucuser@ubuntu-server.example.com`
- 下次直接 `ssh linucuser@ubuntu-server.example.com` 就能免密登录

👉 通俗理解：

SSH 就像钥匙：

- 私钥（你自己保管）
 - 公钥（交给服务器）
- 配对成功 → 以后进门（远程登录）不用每次敲密码。

🔥🔑 本节小结（记忆法）

- **安装：**记住「启动顺序 BIOS/UEFI → 安装菜单 → 时区/语言/分区/软件」
- **关机/重启：**`shutdown -h now`、`shutdown -r now`、`reboot`
- **SSH：**`~/.ssh/id_rsa` 私钥、`~/.ssh/id_rsa.pub` 公钥、`ssh-copy-id`

口诀：

👉 「装机设顺序，开关靠 `shutdown`，远程用 `SSH`」

◆ 1.2 仮想マシン・コンテナの概念と利用

1.2.1 仮想マシンとコンテナ

🔥 概念：

- 过去 → 一台物理服务器（物理机）上直接装 OS → 用作 Web 服务器、DB 服务器
- 现在 → 在物理硬件上跑 **虚拟化软件**，创建出多个 **仮想マシン（虚拟机，VM）** 或 **コンテナ（容器）** 来运行服务

仮想マシン（VM）

- **实现方式：**依靠 **ハイパーバイザー（Hypervisor）**

◆ 1. ハイパーバイザーとは？

- 中文：虚拟机管理程序（VMM, Virtual Machine Monitor）
- 功能：把 **物理硬件**（CPU、内存、磁盘、网卡...）虚拟化，然后分配给不同的 VM。
- 位置：运行在硬件与虚拟机之间，负责“骗”虚拟机，让它们觉得自己独占一整台电脑。

👉 简单理解：

ハイパーバイザー就是“物业公司”。

- 大楼 = 物理服务器
- 公寓 = VM
- 物业公司（Hypervisor）负责给每个公寓分配水电煤气（CPU、内存、存储），保证互不干扰。

◆ 2. 分类（考试常考！）

① Type 1: ネイティブ型 / ベアメタル型

- Hypervisor 直接跑在物理硬件上
- 性能最好，常用于服务器虚拟化
- 例：
 - VMware ESXi
 - Microsoft Hyper-V (Server Core)
 - KVM (Linux 内核模块)

👉 就像物业公司直接建在大楼里面，效率高。

② Type 2: ホスト型

- Hypervisor 跑在已有的操作系统上
- 适合桌面虚拟化，灵活但性能稍差
- 例：
 - VirtualBox
 - VMware Workstation / Fusion

👉 就像在公寓里再隔一层租给别人，中间多了一层管理，速度慢一点。

◆ 3. 和 Container 的区别

- VM → 需要 **Hypervisor** 提供完整虚拟硬件和内核
- Container → 不需要 Hypervisor，只需要共享宿主机的内核

👉 所以 **Hypervisor = VM 的必要条件**，而 **容器不需要 Hypervisor**。

◆ 4. 考试记忆法

Type 1 = 裸金属（性能最好），Type 2 = OS 上跑（灵活但慢）

- **运行结构：**
 - 底层是物理硬件
 - 上面有宿主机 OS (Host OS)
 - 再上面跑 Hypervisor
 - Hypervisor 上可以创建多个 **ゲスト OS (Guest OS)**
- **例子：**在一台 CentOS 主机 (Host OS) 上跑一个 Ubuntu Guest OS 和一个 Debian Guest OS。
- **特点：**每个 VM 都是「完整的操作系统」，相互隔离，就像多台独立服务器。

👉 **通俗理解：**虚拟机 = 在一台电脑里装了很多台完整的电脑。

コンテナ (容器)

- **实现方式：**共享宿主机的 **OS 内核**，通过 namespace / cgroup 技术隔离
- **运行结构：**

- 宿主机 OS 上直接用容器引擎（Docker, Podman）运行
- 每个容器只包含运行所需的 **应用 + 依赖环境**，不需要一个完整的 Guest OS
- **特点**：轻量级，启动快，资源占用少。
- **例子**：在一台 CentOS 宿主机上跑 Docker 容器，容器里有 Nginx、MySQL、Python 应用。

👉 **通俗理解**：容器 = 宿主机里的「独立应用小盒子」，它不像 VM 那样自带整台电脑，只带运行应用需要的东西。

其实很多人学 Linux 或云计算时都会混淆 **仮想マシン (VM)** 和 **コンテナ (容器)**。我帮你用通俗的例子来解释：

◆ 1. 仮想マシン (Virtual Machine, VM)

- **原理**：在一台物理机上，通过 **ハイパーバイザー (Hypervisor, 虚拟化管理程序)** 模拟出多台完整的计算机。
- **结构**：每个 VM 都有 **完整的 OS (操作系统内核)**。
- **资源**：CPU、内存、磁盘都是虚拟分配的，好像真的有好几台机器。

VM (虚拟机) = 独门独户的公寓

- 每个 VM 都有自己的水电系统（操作系统内核），彼此完全独立，隔音（隔离性）也好，就是贵（资源消耗大，启动慢）。

◆ 2. コンテナ (Container)

- **原理**：不是模拟整台电脑，而是共享 **宿主机的 OS 内核**，在上面隔离出多个独立的运行环境。
- **结构**：每个容器只带上必要的运行环境（程序、库文件），没有自己单独的内核。
- **资源**：隔离性比 VM 弱一些，但轻量、启动快。

Container (容器) = 合租公寓里的独立房间

- 大家共用一个厨房、厕所（宿主机内核），所以更轻便省资源，启动也快，但隔音（隔离性）没有独门独户那么强。

👉 考试时候记忆口诀：

- **VM：各自有内核 → 重但隔离强**
- **容器：共享内核 → 轻但隔离弱**

◆ 3. 对比表

項目	仮想マシン (VM)	コンテナ (Container)
内核	每个 VM 自带一套 OS 内核	所有容器共享宿主机的内核
启动速度	慢 (要启动完整的 OS)	快 (秒级, 直接用宿主机内核)
占用资源	重 (每个 VM 至少要几百 MB ~ GB)	轻 (通常几十 MB)
隔离性	强 (几乎像真正的独立电脑)	弱一些 (共享内核)
应用场景	需要不同操作系统、强隔离的场景	微服务、快速部署、大规模伸缩场景

◆ 4. 简单记忆法

- **VM = 公寓**：独立，完整，重
- **Container = 隔断房间**：共享内核，轻量，快

👉 考试小技巧：

如果题目出现“**内核是否共享**”，就能快速区分：

- **VM**：不共享，每个都有自己的内核
- **Container**：共享宿主机内核

1.2.2 KVM仮想マシンの操作

◆ 1. 什么是 KVM?

- **KVM (Kernel-based Virtual Machine)**
= 基于内核的虚拟机

- 它不是单独的软件，而是 **Linux 内核里的一个模块**。
- 只要开启这个模块，Linux 自己就能当 Hypervisor，用来跑虚拟机。

👉 打个比方：

原来：Linux = 普通住户，只能在自己的房间里生活（跑应用）。

加了 KVM 模块：Linux 拿到“物业管理证”（Hypervisor 功能），就从住户升级成了 **整栋大楼的物业经理**。

结果：它不仅能自己住，还能在大楼里划出好多间房（虚拟机）租给别人住。

💎 2. 为什么需要 CPU 的虚拟化指令？

背景：虚拟化为什么需要特殊指令？

- 普通情况下，**操作系统**要直接和 CPU 打交道。
- 如果我想在一台电脑里跑一个“假的电脑”（虚拟机），那虚拟机的操作系统也要去访问 CPU。

👉 问题来了：

虚拟机里的 OS 觉得自己是“老大”，它发的 CPU 指令会和宿主机的 OS 冲突。

这就像两个司机同时抢一台方向盘，车直接翻了。

CPU 虚拟化指令干嘛的？

于是 CPU 厂商（Intel、AMD）就在 CPU 里加了一组 **专门支持虚拟化的指令集**：

- **Intel** → VT-x（标志是 `vmx`）
- **AMD** → AMD-V（标志是 `svm`）

这些指令的作用就是：

👉 告诉 CPU：“这是虚拟机发来的命令，别和宿主机的混淆，交给 Hypervisor 来调度。”

这样虚拟机就能正常跑，性能也接近真机。

怎么用（从用户角度）

你不用直接写这些指令（太底层了，只有编译器/Hypervisor 会调用）。

你要做的就是：

1. BIOS/UEFI 开启虚拟化支持

- 一般在 BIOS 里有选项：
 - Intel → “Intel VT-x” 或 “Intel Virtualization Technology”
 - AMD → “SVM Mode” 或 “AMD-V”
- 默认有些电脑是关掉的，需要手动开。

2. 在 Linux 里确认支持情况

3. `lscpu | grep Virtualization`

```
grep -E 'vmx|svm' /proc/cpuinfo
```

- 会在 CPU 的功能标志里搜索 `vmx` 或 `svm`。
- 结果更底层、更原始，比如：

```
flags: fpu vme de pse tsc msr pae mce cx8 apic sep ... vmx ...
```
- 只要在 flags 里看到了 `vmx` 或 `svm`，就说明支持虚拟化。

4. 上层软件（KVM、VMware、VirtualBox）就会自动利用这些指令来跑虚拟机。

👉 你只要开关机，Hypervisor 会帮你调用 CPU 的虚拟化功能。

通俗比喻

- **没有虚拟化指令的 CPU：**
 - = 一个车只有一个方向盘，司机（宿主机）坐上去后，别人没法开。
- **有虚拟化指令的 CPU：**
 - = 方向盘上加了“驾驶权限分配系统”，Hypervisor 可以让不同司机（虚拟机）轮流开车，而且不打架。

◆ 3. 怎么确认 CPU 支不支持虚拟化?

在 Linux 里有两个常用方法:

方法1: 用 `lscpu` 命令

```
lscpu | grep Virtualization
```

结果可能会显示:

- `Virtualization: VT-x` (说明是 Intel 并支持)
- `Virtualization: AMD-V` (说明是 AMD 并支持)

方法2: 查 CPU 的特性标志

```
grep -E 'vmx|svm' /proc/cpuinfo
```

- **vmx** = Intel 的虚拟化指令 (VT-x)
- **svm** = AMD 的虚拟化指令 (AMD-V)

只要能查到其中之一, 就说明你的 CPU 支持虚拟化。

👉 记忆口诀:

- **vmx = Intel**
- **svm = AMD**

◆ 4. 总结 (考试重点)

1. **KVM 是 Linux 内核里的虚拟化功能**
2. **必须 CPU 支持虚拟化指令 (Intel VT-x / AMD-V)**
3. **检查方法:**

- `lscpu | grep Virtualization`
- `grep -E 'vmx|svm' /proc/cpuinfo`

📌 相关术语:

- Virtio: → para-virtualized I/O 驱动, 加速 VM 的网卡/磁盘。
👉 考点: **让 VM 的性能接近真机。**
- ブリッジネットワーク: VM 的虚拟网卡和宿主机的物理网卡桥接 → VM 拥有和宿主机同一网段的 IP。
像是 VM 变成了“局域网里的一台独立主机”。
- ネステッドページング: → CPU/内存虚拟化的加速技术, 减少内存访问开销。
👉 考点: **提升 VM 内存访问性能。**

📌 相关程序:

- `kvm`: KVM 核心模块
- `kvm_intel` / `kvm_amd`: Intel/AMD 模块
- `virsh`: CLI 管理工具 (基于 libvirt)
- `virt-manager`: GUI 管理工具
- `virt-install`: CLI 创建/安装虚拟机
- `libvirt`: 底层库, 提供统一 API (VMware/KVM/Xen 都能通过它管理)

高频总结口诀 (应试用)

1. 模块:

- Intel → `kvm_intel`
- AMD → `kvm_amd`

2. 性能加速:

- I/O → Virtio

- メモリ → ネステッドページング

3. 网络:

- 同网段 → Bridge

4. 管理工具:

- CLI 管理 → virsh
- GUI 管理 → virt-manager
- CLI 安装 → virt-install
- API → libvirt

1.2.3 virshコマンドを使った仮想マシンの操作

🔥 常用命令:

```
virsh list --all          # 显示所有虚拟机
virsh start vm-ubuntu     # 启动虚拟机
virsh shutdown vm-ubuntu  # 关闭虚拟机 (正常)
virsh destroy vm-ubuntu   # 强制关闭 (可能损坏文件系统)
virsh console vm-ubuntu   # 连接到虚拟机控制台
```

👉 记忆法:

- start = 开机
- shutdown = 正常关机
- destroy = 拔电源强制关机
- console = 进入控制台

1.2.4 dockerコマンドを使ったコンテナの操作

🔥 核心概念:

- Docker イメージ (镜像): 应用的“模版”
- コンテナ (容器): 运行中的实例
- Docker レジストリ: 镜像仓库 (官方: Docker Hub)

🔥 常用命令:

```
docker pull centos:7      # 拉取镜像
docker images             # 查看镜像
docker run centos:7 cat /etc/hostname # 基于镜像启动一次性容器
docker ps                 # 查看运行中的容器
docker ps -a              # 查看所有容器 (包括已停止的)
docker rm <容器ID>        # 删除容器
docker run -it --name test centos:7 /bin/bash # 交互式启动
docker attach <容器名>    # 进入正在运行的容器
docker start <容器ID>     # 启动容器
docker stop <容器ID>      # 停止容器
```

👉 通俗理解:

- 镜像 (Image) = 应用快照
- 容器 (Container) = 运行时的进程
- Docker = 像“工厂”: 拿模版 (镜像) 生产出多个容器 (实例)。

🏆🔑 本节总结 (考试考点)

1. 虚拟机 vs 容器区别:

- VM: 完整 OS, 重, 隔离强
- 容器: 共享内核, 轻, 启动快

2. KVM 必考点:

- VT-x / AMD-V → CPU 硬件虚拟化支持
- `virsh start/shutdown/destroy`

3. Docker 必考点:

- `docker pull/run/ps/start/stop/rm`
- 镜像 vs 容器的概念

◆ 1.3 ブートプロセスとsystemd

1. 系统启动流程

当你按下电源按钮以后，Linux 是这样启动的（就像开一台车的流程）：

1. **BIOS/UEFI**：检查硬件能不能用（相当于启动前自检）。
2. **Boot Loader（引导加载器）**：找到并加载 Linux 内核。常见：GRUB2。
3. **Kernel（内核）**：接管控制，加载驱动，让硬盘/显卡/网卡可用。
4. **systemd（PID=1）**：启动各种服务（网络、用户登录界面），最后让你能登录系统。

👉 考试常问：启动顺序是什么？

答案就是：

BIOS/UEFI → Boot Loader → Kernel → systemd

2. systemd 的 Boot Target（启动模式）

Linux 可以有不同的启动模式，就像汽车有「运动模式」「省油模式」一样。

- **multi-user.target** → 纯命令行模式（CUI）
- **graphical.target** → 图形界面模式（GUI）
- **rescue.target** → 单用户模式，用来修复系统
- **reboot.target** → 重启
- **poweroff.target** → 关机

🔧 常用命令：

- 查看默认启动模式
- `systemctl get-default`
- 👉 如果输出 `graphical.target`，说明开机默认进 GUI
- 👉 如果输出 `multi-user.target`，说明开机默认进命令行
- 修改默认启动模式为命令行：
- `sudo systemctl set-default multi-user.target`
- 修改默认启动模式为图形界面：
- `sudo systemctl set-default graphical.target`
- 临时切换到单用户模式（不用重启）：
- `sudo systemctl isolate rescue.target`

3. 关机与重启命令

Linux 下关机、重启有两种方式：

✅ 用 `systemctl`（推荐）

- 立即关机
- `sudo systemctl poweroff`
- 立即重启
- `sudo systemctl reboot`

✅ 用传统 shutdown

格式：

```
shutdown [选项] 时间 [消息]
```

常见：

- 立即关机
- `sudo shutdown -h now`
- 立即重启
- `sudo shutdown -r now`
- 5分钟后关机，并通知用户
- `sudo shutdown -h +5 "System will shutdown in 5 minutes"`
- 只发警告，不关机
- `sudo shutdown -k now "Please logout immediately!"`
- 取消已安排的关机
- `sudo shutdown -c`

考试中常见的题型

? 问：如何查看当前系统默认启动模式？

✅ 答：

```
systemctl get-default
```

? 问：如何把系统改成下次启动进入命令行模式？

✅ 答：

```
sudo systemctl set-default multi-user.target
```

? 问：如何立刻重启系统？

✅ 答：

```
sudo shutdown -r now
```

(或 `systemctl reboot`)

🗝️ 记忆方法

1. 启动流程口诀：

BIOS → **B**oot Loader → **K**ernel → **S**ystemd
(记成 BBKS)

2. target 对照表 (必须记住!) :

- multi-user.target = 命令行模式
- graphical.target = 图形界面模式
- rescue.target = 单用户修复
- reboot.target = 重启
- poweroff.target = 关机

3. shutdown 参数口诀：

-h = halt (关机)
-r = reboot (重启)
-k = kick (只发通知，不关机)
-c = cancel (取消关机)

◆ 1.4 プロセスの生成、監視、終了

1.4.1 プロセスの監視 (进程的监视)

① プロセスとは？

- プロセス = OS 管理下正在运行的程序的基本单位。
- 程序文件在硬盘（HDD/SSD）上 → 运行时会被 读入内存 并交给 CPU 执行 → 这就是进程。
- 可以理解成：进程 = “正在执行中的程序” 或者 “可以执行的状态的程序”。

📌 例子：
服务器上的进程可能长时间存在（例如 Apache、sshd），
而命令行里的 `ls` 可能一瞬间就结束。

② プロセスの表示： `ps` コマンド

- 作用：显示当前正在运行的进程。

常用选项（表 1-6）：

オプション	説明
<code>a</code>	显示所有用户的进程
<code>f</code>	以树状结构显示父子关系
<code>u</code>	显示用户名
<code>x</code>	显示无控制终端的守护进程
<code>-e</code>	显示所有进程（= <code>ax</code> ）
<code>-l</code>	显示详细信息
<code>-p PID</code>	指定 PID 的进程
<code>-C プロセス名</code>	指定名字的进程
<code>-W</code>	长行换行显示

⚠ 注意：

- `ps` 的选项里有的写 `-`，有的不写 `-`，行为可能不同，要小心考试。

③ 例子

```
$ ps
PID  TTY    TIME  CMD
1226 pts/0    00:00  bash
1275 pts/0    00:00  ps
```

👉 当前终端里只有 `bash` 和 `ps` 本身。

```
$ ps ax
PID TTY  STAT TIME COMMAND
1 ?   Ss   0:00 /usr/lib/systemd/systemd --system
2 ?   S    0:00 [kthreadd]
3 ?   S    0:00 [rcu_gp]
...
```

👉 `ax` 会列出所有进程，包括守护进程。

④ プロセスの継続監視： `top`

- `top` 会持续更新进程状态（动态监控）。
- 显示内容：
 - 第 1 行：系统运行时间、用户数、load average
 - 第 2 行：任务数（运行、睡眠、停止、僵尸）
 - 第 3 行：CPU 使用情况
 - 第 4~5 行：内存、swap 使用情况
 - 之后：各进程的 PID、用户、优先级、CPU 使用率等

📌 考点：

- `top` 默认按 CPU 使用率排序
- 退出 `top` 用 `Q` 键

⑤ システム稼働状況の確認: `uptime`

```
$ uptime
22:38:53 up 1:03, 2 users, load average: 0.00, 0.01, 0.05
```

显示: 系统运行时长、登录用户数、平均负载 (1分钟、5分钟、15分钟)。

load average = 平均等待 CPU 的进程数。

- 一般来说, 值大于 **CPU 核心数** 就说明系统过载。

⑥ 親プロセスと子プロセス

- 一个进程执行另一个 → 新的叫 **子进程**, 原来的叫 **父进程**。
- 可以用 `ps tree` 命令查看层级结构。

例:

```
$ ps tree
systemd├─NetworkManager├─dhclient
        │└─2*[{NetworkManager}]
        ├─anacron
        ├─auditd└─{auditd}
        └─crond
        ...
```

👉 `ps tree` 把父子关系画成树状, 非常直观。

Signal 信号

- `kill -l`: 列出全部信号与编号。
- 表1-7 主要信号 (背)**
 - HUP(1)**: 挂起/重新读取配置 (常用于**让守护进程重读配置**)
 - INT(2)**: 键盘中断 (≈**Ctrl+C**)
 - KILL(9)**: **强制结束** (不可捕获/忽略, 最后手段)
 - TERM(15)**: **正常结束 (默认)**
 - CONT(18)**: 继续已停止的进程
 - STOP(19)**: 立即**暂停** (内核级停止)

当不指定信号执行 `kill` 就意味着⇒ 发送 `TERM(15)`。

1.4.2 プロセスの終了

在 Linux 里, 如果你想让一个进程停掉, 主要的手段就是 `kill` 命令。

⚠ 注意: 这里的 `kill` 不是“秒杀”, 而是“发送一个信号 (signal)”给进程。

`kill` 的几种写法

书上列了三种写法, 其实都是一样的意思, 只是表达方式不同:

```
kill -[信号名或编号] PID
kill -s [信号名或编号] PID
kill -SIG信号名 PID
```

👉 默认情况下, 你只写 `kill PID`, 系统会发 **TERM (15)** 这个信号。也就是说, 这是“请你优雅退出一下”的意思。

示例

比如, PID 是 560 的进程:

```
kill 560          # 默认发 TERM
kill -15 560      # 明确发 TERM
kill -s 15 560    # 同上
kill -TERM 560    # 用名字写
kill -SIGTERM 560 # 更完整写法
```

👉 如果一个进程很顽固, 用 `TERM` 信号杀不掉, 你就要上最后的武器:


```
kill -KILL 560
```

这会直接把进程干掉，不留机会收尾。

但正如书里提醒的：**不要随使用 KILL**。因为这样可能会造成文件没正常保存、系统出问题。它应该是“最后手段”。

一次杀多个

你也可以一次性指定多个 PID：

```
kill 570 571
```

1.4.2 的延伸：找 PID

要 kill 进程，你得知道 PID。那怎么查呢？

```
pgrep
```

这是一个很方便命令，能直接按进程名查 PID。

书上的表 1-8 列了几个选项：

- `-U 用户名`：指定某个用户的进程
- `-G 组名`：指定某个组
- `-n`：最新的一个
- `-o`：最老的一个
- `-l`：同时显示进程名

例子：

```
pgrep bash
pgrep -U linucuser
```

killall & pkill

```
killall
```

如果你懒得查 PID，直接对名字下手：

```
killall vim
```

这会对所有名字叫 **vim** 的进程发默认的 TERM。

```
pkill
```

`pkill` 和 `killall` 很像，不过它支持更多筛选条件（书上表 1-9）：

- `-U 用户名`
- `-G 组名`

例子：

```
pkill -U linucuser vim
```

意思就是：只杀掉 linucuser 这个用户开的 vim，别的用户不受影响。

1.4.3 ジョブ管理

1. 什么是「ジョブ」

- **ジョブ**：在 **シェル** 中执行的一条命令，或者通过管道连接起来的一组命令。
👉 在终端里，你一次敲的那条“任务”，就是一个ジョブ。

例子：

- `ls -l` → 一个ジョブ
- `ls -l | grep log` → 一个ジョブ（但由两个进程组成）

2. フォアグラウンド と バックグラウンド

- **フォアグラウンドジョブ**：前台作业，会占住终端，你必须等它结束才能做别的。
 - 例：`sleep 60` → 这 60 秒终端都被锁着。

- **バックグラウンドジョブ**: 后台作业, 在命令后加 `&`, 就能一边执行这个作业, 一边在终端做别的。

- ```
sleep 120 &
```

```
[1] 14779
```

  - `[1]` = ジョブ番号 (作业号)
  - `14779` = PID (进程号)

👉 前台 = 正在舞台上表演, 后台 = 悄悄在后台排练。

### 3. 查看ジョブ: `jobs`

- `jobs` 命令会列出当前シェル管理的所有ジョブ。
- 状态有:
  - **Running** (后台执行中)
  - **Stopped** (暂停中)
- 标记符号:
  - `+` = 当前ジョブ (カレントジョブ)
  - `-` = 前一个ジョブ

例子:

```
$ jobs
```

```
[1]+ Stopped vim test.txt
```

```
[2]- Running sleep 100 &
```

👉 就像一个“清单”, 告诉你哪个任务在跑, 哪个停下了。

### 4. 前后台切换

有时候你在前台跑的任务, 想要丢后台继续, 或者从后台拉回前台。

1. **Ctrl+Z**: 把前台任务先暂停 (Stopped 状态)。
2. `bg %作业号`: 让它在后台继续执行。
3. `bg %1`
4. `fg %作业号`: 把后台的任务拉回前台。
5. `fg %1`

👉 `Ctrl+Z` = 暂停演出; `bg` = 让它在后台继续; `fg` = 再次把它拉回舞台。

### 5. ログアウト後も続ける: `nohup`

普通后台作业如果你退出登录 (logout), 就会被系统发信号杀掉。

这时可以用 **nohup** (no hang up) :

```
nohup updatedb &
```

- 这样即使你退出登录, ジョブ也不会消失。
- 输出会写到一个 `nohup.out` 文件。

👉 `nohup` 就像给后台任务加了“护身符”, 你走了, 它也能继续跑。

#### 🔪 ✅ 小结 (记忆要点)

- ジョブ = Shell 里的任务集合
- **后台执行**: 命令 + `&`
- **jobs**: 查看状态 (Running/Stopped, +/-)
- **Ctrl+Z**: 暂停
- **bg %n**: 后台继续
- **fg %n**: 前台恢复

- **nohup**: 退出后也能继续

#### 1.4.4 フォアグラウンドとバックグラウンド —— 前后台切换

有时候，你已经在前台跑了一个任务，不想中断，但又需要释放终端。

步骤：

1. **Ctrl+Z**: 先把进程挂起（暂停）。
2. `bg %作业号` : 让它去后台继续跑。
3. `fg %作业号` : 再把后台任务拉回前台。

书里的例子：

```
bg %1
fg %1
```

#### ▶ 1.4.5 端末の活用 (tmux)

### ◆ 1.5 デスクトップ環境の利用

#### 1.5.1 X Window Systemの構成

这一节的核心就是：**Linux 内核本身并不带 GUI（图形界面）功能**，所以要靠一个额外的系统——**X Window System (X11)** 来实现。

#### ◆ Linux 本身没有 GUI

Linux kernel 只管硬件和底层（进程、内存、驱动），**不自带 GUI 功能**。  
所以需要额外的系统来实现 GUI → **X Window System (简称 X, X11)**。

教材里明确写：

Linuxカーネルには、GUIを実現する機能は備わっていません。

#### ◆ X Window System 的历史与实现

- **X Window System (X11)** → 很早就有的机制，用于提供图形界面。
- 现在 Linux 里的主要实现有两个：
  - nq. X.Org**: 从 UNIX 系统时代传下来的老牌实现，现在依然主流。
  - nr. Wayland**: 新开发的替代方案，设计更现代，减少中间层，效率更高。

所以现在很多 Linux 桌面环境是基于 **X.Org 或 Wayland** 来跑的。

#### ◆ X Server 和 X Client (图 1-18)

书里画了个图：

- **X サーバ (X Server)**
  - 管理显示器、显卡、键盘、鼠标等硬件。
  - 就像“后台服务”，帮应用程序画东西、接收输入。
- **X クライアント (X Client)**
  - 就是用户的应用程序（浏览器、办公软件、终端等等）。
  - 它们自己不会直接画图，而是把“绘图请求”交给 X Server 来完成。

👉 关系：

X Server 负责画 → X Client 只管告诉它要画什么。

**即使 X Server 和 X Client 在不同机器上，也可以通过网络来配合工作。**

◆ ウィンドウマネージャ (Window Manager)

教材解释：  
X Server 只管把应用窗口画出来，但这些窗口的 **外观（边框、菜单、图标）**，以及你能不能拖动、最小化，这些都需要 **Window Manager** 来控制。

🚩 代表性例子（表 1-10）：

- **twm**：最基础的窗口管理器
- **FVWM**：轻量级
- **Enlightenment**：高度可定制
- **Metacity**：GNOME 2 默认
- **Mutter**：GNOME 3 默认
- **Fluxbox**：轻量且可定制
- **Compiz**：炫酷的 3D 效果
- **KWin**：KDE 的默认窗口管理器

⚠ 注意：Window Manager 不同，用户的 UI（界面外观）可能会完全不一样。

---

◆ 統合デスクトップ環境 (Integrated Desktop Environment)

仅有 Window Manager 还不够，用户需要更完整的体验，比如文件管理器、控制面板、预装应用等。  
→ 所以就有 **統合デスクトップ環境（综合桌面环境）**。

代表性的有：

- **GNOME**
  - 被 Red Hat Enterprise Linux, CentOS, Fedora, Ubuntu 等作为标准桌面。
- **KDE (KDE Plasma)**
  - 被 openSUSE, Kubuntu 等作为标准桌面。

(图 1-20 是 GNOME 界面，图 1-21 是 KDE 界面)

---

◆ GNOME vs KDE Plasma 对比（表 1-11）

书里还对比了这两个桌面环境自带的软件：

| 类型    | GNOME          | KDE Plasma |
|-------|----------------|------------|
| 文本编辑器 | gedit          | KEdit      |
| 终端    | GNOME Terminal | Konsole    |
| 文件管理器 | Nautilus       | Dolphin    |
| 显示管理器 | GDM            | SDDM       |
| 窗口管理器 | Mutter         | KWin       |

👉 你可以看到，两大桌面环境都有自己的一套“全家桶”，覆盖文本编辑、文件管理、窗口管理等。

---

◆ 轻量级桌面环境

GNOME 和 KDE 功能强大，但对 **CPU、内存要求高**，老电脑可能跑不动。  
所以也有更轻量的选择，比如：

- **Xfce**
- **LXDE**
- **MATE**

(图 1-22 给了一个 Xfce 的例子)

---

◆ ディスプレイマネージャ (Display Manager)

最后一块：用户登录界面的管理。

- 它负责显示登录界面，让你输入用户名/密码登录。

- 代表：
  - **XDM**: X.Org 自带的
  - **GDM**: GNOME 的登录管理器
  - **SDDM**: KDE Plasma 的登录管理器

🔗 图 1-23 是 Ubuntu 的 GDM 登录界面。

🔗 图 1-24 显示了用户在登录时可以切换桌面环境（比如 Fluxbox、Ubuntu、Xfce）。

## ◆ Wayland 的出现（图 1-25）

教材最后提到：

- X Window System 太老了，架构复杂，在现代电脑上已经显得笨重。
- 所以出现了 **Wayland**，它让 Window Manager 直接接管显示和输入（不再通过 X Server 这个中间层）。
- 这样可以更高效，但目前并不是所有应用都完全兼容 Wayland。

### 📁🎯 小结（1.5.1 的要点）

1. **Linux kernel 不带 GUI**，需要 X Window System。
2. **X Server** 管理硬件，**X Client** 是应用程序。
3. **Window Manager** 决定窗口外观和交互。
4. **Desktop Environment** 提供完整体验（GNOME / KDE 等）。
5. GNOME、KDE Plasma 是最常见的桌面环境，各有一套应用生态。
6. 轻量替代有 Xfce、LXDE、MATE。
7. **Display Manager** 管理登录界面（GDM、SDDM 等）。
8. **Wayland** 是新一代替代方案，效率更高，但兼容性还在逐步完善。

## 1.5.2 GUIの起動

这一节讲的是：**如何启动 GUI（图形界面）**。

### ◆ 使用 startx 启动 GUI

在 Linux 里，如果已经安装好了 **X Window System**，并且配置正确，可以在 **控制台（文字界面）** 下直接输入：

```
$ startx
```

- 作用：启动 X Window System
- 如果你退出 GUI（比如点击注销/logout），就会回到文字控制台（Console）

👉 所以 **startx** 其实是手动启动 GUI 的命令。

### ◆ 使用 systemctl 切换到 graphical.target

现在大部分 Linux 发行版（如 CentOS7, Ubuntu, Fedora 等）都使用 **systemd** 管理服务和启动流程。

所以更常见的做法是用 systemctl 来切换到图形界面：

```
systemctl isolate graphical.target
```

解释：

- **isolate**: 切换到指定的 target（类似 runlevel）
- **graphical.target**: 表示图形模式（GUI 登录界面）

👉 执行完这个命令，系统就会进入图形界面。

### ◆ 和前面学过的 multi-user.target 对比

在 **1.3.2 ブートターゲット** 我们讲过：

- **multi-user.target** → 文字界面（CUI 登录）
- **graphical.target** → 图形界面（GUI 登录）

所以 `systemctl isolate graphical.target`，其实就是：

“临时切换到图形界面”。

如果你想要每次开机默认就是图形界面，而不是文字界面，需要用：

```
systemctl set-default graphical.target
```

(这一点在 1.3 已经讲过，这里是复习和联系)

## ◆ 注意事项

教材里特别提醒：

附带的虚拟机练习环境（教材用的环境）里，**没有安装 X Window System。**

所以书里给的命令（`startx`、`systemctl isolate graphical.target`）在那个环境里没法演示。

考试只需要理解命令的作用，不要求真的跑 GUI。

## 小结

1. **startx** → 在文字界面下手动启动 GUI。
2. **systemctl isolate graphical.target** → 用 `systemd` 切换到图形界面。
3. **graphical.target** 和 **multi-user.target** 对应 GUI / CUI 模式。
4. 教材环境不带 GUI，考试只要求会写命令、理解原理。

## ▶ 1.5.3 ネットワーク経由でのXの利用

## 第2章 ファイル・ディレクトリの操作と管理

### 2.1基本的なファイル管理の実行

#### 2.1.1 ファイル操作コマンド

这一节的内容主要是介绍在 Linux 里 **最基本的文件/目录操作命令**，比如显示文件、复制、移动、删除、新建等。考试和实际工作里都很常用。

这一节主要讲 **日常 Linux 操作中最常用的文件与目录管理命令**。

比如：

- 查看目录内容 → `ls`
- 复制文件/目录 → `cp`
- 移动或改名 → `mv`
- 创建目录 → `mkdir`
- 删除文件/目录 → `rm` , `rmdir`
- 修改时间戳 → `touch`
- 查看文件种类 → `file`

第一部分先从 `ls` 命令开始。

## `ls` 命令

作用：**列出目录中的文件和子目录**，或者显示指定文件的属性。

基本格式：

```
ls [选项] [文件名或目录名]
```

 书上的 **表2-1** 给了主要选项：

| 选项                   | 说明                              |
|----------------------|---------------------------------|
| -a, --all            | 显示所有文件，包括以 . 开头的隐藏文件            |
| -A, --almost-all     | 显示隐藏文件，但不显示 .(当前目录) 和 ..(上级目录)  |
| -d, --directory      | 仅显示目录本身的信息，而不是目录下的内容            |
| -F, --classify       | 在输出结果后加标记：目录/，可执行文件*，符号链接@      |
| -i, --inode          | 显示文件的 inode 号（文件系统里的唯一编号）       |
| -l, --format=long    | 详细列表（权限、所有者、大小、时间等）             |
| -r, --reverse        | 反向排序                            |
| -R, --recursive      | 递归显示子目录                         |
| -t, --sort=time      | 按修改时间排序                         |
| -h, --human-readable | 人类可读的方式显示大小（配合 -l 使用，比如 KB, MB） |

1. 示例

• 显示目录内容

```
ls /home/linucuser
```

👉 输出：只列出 `exdir` 目录。

• 显示目录本身的信息（不是里面的文件）

```
ls -ld /home/linucuser
```

👉 输出：会列出 `/home/linucuser` 的权限、所有者、大小、修改时间。

• 显示隐藏文件

```
ls -A
```

👉 输出中包含 `.bashrc`、`.profile` 这些隐藏文件。

• 详细显示

```
ls -lA
```

👉 输出会有权限（`drwxr-xr-x`）、所有者、大小、时间等详细信息。

2. 小结

- `ls` 是最常用的命令之一，必须熟练掌握。
- `-l` 和 `-a` 经常配合使用，写成 `ls -la`。
- 如果只是要看目录本身而不是内容，用 `ls -d`。
- 隐藏文件记住是 **点号开头的文件**（`.`）。

`cp` 命令

作用：**复制文件或目录**。

基本格式：

```
cp [选项] 源文件 目标文件
cp [选项] 源文件 目标目录
```

📖 教科书 **表2-2** 给的主要选项：

| 选项                  | 说明                                         |
|---------------------|--------------------------------------------|
| -f, --force         | 直接覆盖同名文件（不询问）                              |
| -i, --interactive   | 覆盖同名文件时会提示确认                               |
| -p, --preserve      | 保留原文件属性（所有者、权限、时间戳）                        |
| -R, -r, --recursive | 递归复制整个目录                                   |
| -d                  | 保留符号链接为链接形式（不复制其指向的内容）                     |
| -a, --archive       | 归档模式，尽量保持源文件的所有结构和属性，相当于 <code>-dpR</code> |

1. 示例

• 复制文件

```
cp /etc/hosts hosts.org
```

👉 把 `/etc/hosts` 文件复制为 `hosts.org`。

- 复制目录中的所有 `.conf` 文件

```
cp /etc/modprobe.d/*.conf /tmp
```

👉 把所有 `.conf` 文件复制到 `/tmp` 目录下。

- 递归复制整个目录

```
cp -r /etc/cron.d .
```

👉 把 `/etc/cron.d` 目录完整复制到当前目录。

⚠️ 注意：复制目录必须加 `-r`，否则会报错。

## 2. 注意事项

书上特别强调（这里有个「ここが重要」的提示）：

- 复制目录时必须用 `-r`。
- 如果想保留文件属性（权限、时间戳等），需要加 `-p`。
- 实际工作中经常用 `-a`（归档模式），相当于完整备份。

## 3. 小结

- 文件复制 → `cp 源 目标`
- 目录复制 → `cp -r 源目录 目标目录`
- 常用组合 → `cp -a`（相当于完整拷贝，包括属性和符号链接）

## mv 命令

作用：**移动文件/目录** 或者 **重命名文件/目录**。

（可以理解为 Windows 里的“剪切+粘贴”，或者“改名”）

基本格式：

```
mv [选项] 源文件 目标文件
```

```
mv [选项] 源文件 目标目录
```

## 教科书 表2-3 的主要选项

| 选项                             | 说明                  |
|--------------------------------|---------------------|
| <code>-f, --force</code>       | 直接覆盖目标文件，不询问        |
| <code>-i, --interactive</code> | 如果目标已有同名文件，提示确认后再覆盖 |

## 1. 示例

- 移动到目录

```
mv testfile ~
```

👉 把 `testfile` 移动到用户的 home 目录（`~`）。

- 重命名文件

```
mv umasktest umasktest.org
```

👉 把文件名从 `umasktest` 改成 `umasktest.org`。

## 2. 使用技巧

- `mv` 没有 `-r` 这种选项，因为它移动目录时会自动处理整个目录。
- 和 `cp` 的区别是：
  - `cp` 复制 → 源文件还在。
  - `mv` 移动/改名 → 源文件消失，只在新位置/新名字存在。

## 3. 小结



- **移动文件** → `mv file dir/`
- **重命名文件** → `mv oldname newname`
- **覆盖时要小心**，建议加 `-i` 防止误操作。

`mkdir` 命令

作用：**新建目录（文件夹）**。

基本格式：

```
mkdir [选项] 目录名
```

教科书 表2-4 的主要选项

| 选项            | 说明                       |
|---------------|--------------------------|
| -m, --mode 权限 | 创建目录时，直接指定访问权限（类似 chmod） |
| -p, --parents | 如果上级目录不存在，就一并创建          |

1. 示例

- **创建一个目录并指定权限**

```
mkdir -m 700 mydir
```

👉 创建一个 `mydir` 目录，权限是 **700**（只有自己能读写执行）。

- **递归创建目录**

```
mkdir top/second/third
```

👉 会报错：因为 `top` 或 `second` 目录不存在。

正确做法：

```
mkdir -p top/second/third
```

👉 `-p` 会自动创建缺失的上级目录，所以能一次性建好 `top → second → third`。

2. 使用技巧

- 如果只是普通建文件夹 → `mkdir 名字`。
- 想控制权限 → 用 `-m`。
- 想批量建目录层级 → 用 `-p`。

3. 小结

- `mkdir` 是“创建目录”的命令。
- **重点记忆**：`-m`（权限）、`-p`（递归创建）。
- `-p` 在实际运维中超级常用，比如一次性建多层路径时。

`rm` 命令

作用：**删除文件或目录**。

（⚠️ 注意：和 Windows 不同，Linux 的 `rm` 不会进“回收站”，删除后很难恢复，所以要小心。）

基本格式：

```
rm [选项] 文件名
```

教科书 表2-5 的主要选项

| 选项                  | 说明                        |
|---------------------|---------------------------|
| -f, --force         | 强制删除，不显示错误信息（即使文件不存在也不报错） |
| -i, --interactive   | 每次删除前提示确认                 |
| -r, -R, --recursive | 递归删除整个目录及子目录              |

1. 示例

• 删除文件

```
rm myfile
```

👉 删除 `myfile` 文件。

• 强制删除文件

```
rm -f myfile
```

👉 即使 `myfile` 不存在，也不会报错。

• 删除目录（错误示范）

```
rm mydir
```

👉 会报错：`rm: cannot remove 'mydir': Is a directory`

正确做法：

```
rm -r mydir
```

👉 使用 `-r` 递归删除目录。

• 结合强制删除

```
rm -rf mydir
```

👉 不提示确认，直接删除整个目录树（⚠️ 非常危险）。

2. 安全提示

- 工作中建议用 `rm -i`，避免误删。
- `rm -rf /` 会直接清空整个系统 → 考试不会考，但这是经典“自杀命令”。

3. 小结

- 删除文件 → `rm file`
- 删除目录 → `rm -r dir`
- 强制删除 → `rm -f`
- 最危险的组合：`rm -rf`，要记住它的破坏性。

`rmdir` 命令

作用：删除空目录。  
(和 `rm -r` 不同，`rmdir` 只能删除完全为空的目录。)

基本格式：

```
rmdir [选项] 目录名
```

教科书 表2-6 的主要选项

| 选项            | 说明                        |
|---------------|---------------------------|
| -p, --parents | 递归删除目录，并删除其上级目录（前提是上级也为空） |

1. 示例

• 删除空目录

```
rmdir emptydir
```

👉 如果 `emptydir` 里面什么都没有，就会被删除。

👉 如果 `emptydir` 里面有文件，会报错：

```
rmdir: failed to remove 'emptydir': Directory not empty
```

• 递归删除目录

```
rmdir -p top/second/third
```

- 👉 如果 `third` 是空的，会先删 `third`，再删 `second`，最后删 `top`。
- 👉 但前提是每一层都必须是空目录。

2. 使用技巧

- 如果目录里有内容，就不能用 `rmdir`，要用 `rm -r`。
- 一般 `rmdir` 使用场景不多，更多是脚本里自动清理空目录。

3. 小结

- 删除空目录 → `rmdir dir`
- 递归删除空目录层级 → `rmdir -p path`
- 如果目录不空 → 用 `rm -r`

`touch` 命令

作用：

1. **更新文件的时间戳**（访问时间、修改时间）。
2. **如果文件不存在，就创建一个空文件。**

基本格式：

```
touch [选项] 文件名
```

教科书 表2-7 的主要选项

| 选项                       | 说明               |
|--------------------------|------------------|
| -a                       | 只修改访问时间 (atime)  |
| -m                       | 只修改修改时间 (mtime)  |
| -t [[CC]YY]MMDDhhmm[.ss] | 手动指定时间 (年月日时分秒)  |
| -r 参照文件                  | 让目标文件的时间戳与参照文件一致 |

1. 示例

- **创建新文件**

```
touch myfile
```

- 👉 如果 `myfile` 不存在，就新建一个空文件。
- 👉 如果已存在，就把它的时间戳更新为当前时间。

- **更新访问时间**

```
touch -a myfile
```

- 👉 只修改 atime（最后访问时间）。

- **更新修改时间**

```
touch -m myfile
```

- 👉 只修改 mtime（最后修改时间）。

- **指定时间戳**

```
touch -t 202410101230.55 myfile
```

- 👉 把 `myfile` 的时间戳改为：  
2024年10月10日 12:30分55秒。

- **复制参照文件的时间戳**

```
touch -r ref.txt myfile
```

- 👉 把 `myfile` 的时间戳改成和 `ref.txt` 一样。

---

## 2. 使用技巧

- `touch` 在脚本里常用于：
    - 创建空文件（作为标记）。
    - 修改时间戳（比如测试日志）。
  - Linux 里时间戳有 3 种：
    - **atime**（访问时间）
    - **mtime**（修改时间，文件内容改变时更新）
    - **ctime**（状态改变时间，权限/所有者变化时更新，不能直接用 `touch` 改）
- 

## 3. 小结

- **无文件** → **创建空文件**。
- **有文件** → **更新时间戳**。
- 重要选项：

- `-a` atime
- `-m` mtime
- `-t` 指定时间
- `-r` 参考文件

### `file` 命令

作用：**显示文件的种类**。

👉 在 Linux 里，文件没有像 Windows 那样依赖扩展名（.txt, .jpg）来判断类型，而是通过内容来识别。

`file` 命令就是用来识别文件实际类型的。

基本格式：

`file 文件名`

---

### a. 示例

- **文本文件**

`file /etc/passwd`

输出：

`/etc/passwd: ASCII text`

👉 表明这是一个 ASCII 文本文件。

---

- **二进制文件**

`file /bin/ls`

输出：

`/bin/ls: ELF 64-bit LSB executable, x86-64, ...`

👉 说明这是一个 64 位 ELF 可执行文件。

---

- **压缩文件**

`file archive.tar.gz`

输出：

`archive.tar.gz: gzip compressed data`

👉 即使你把文件名改成 `.txt`，它依然能识别出压缩格式。

---

### b. 使用技巧

- `file` 不依赖扩展名，而是根据文件头信息来判断。

- 对安全运维来说，能快速识别一个陌生文件到底是可执行程序、文本还是脚本，非常重要。
  - 在考试里，常考点是：
    - `file` 能识别实际文件类型，不仅仅是看扩展名。
- 

#### c. 小结

- `file 文件名` → 显示文件类型。
- 能区分 **文本 / 可执行文件 / 压缩文件 / 脚本** 等。
- 在 Linux 里，扩展名不是决定文件类型的关键。

### 2.1.1 文件操作コマンド 速记表

#### 1. `ls` —— 查看目录内容

- 常用选项
    - `-a` 显示隐藏文件
    - `-l` 长格式显示（权限/所有者/时间等）
    - `-d` 显示目录本身而不是内容
    - `-R` 递归子目录
  - 用法
  - `ls -la /etc`
- 

#### 2. `cp` —— 复制文件/目录

- 常用选项
    - `-r` 递归复制目录
    - `-p` 保留权限/时间戳
    - `-a` 完整归档复制（常用组合，相当于 `-dpR`）
    - `-i` 覆盖前确认
  - 用法
  - `cp -a /etc/hosts /tmp/`
- 

#### 3. `mv` —— 移动/重命名

- 常用选项
    - `-i` 覆盖前确认
    - `-f` 强制覆盖
  - 用法
  - `mv file1 /tmp/`  
`mv oldname newname`
- 

#### 4. `mkdir` —— 创建目录

- 常用选项
    - `-p` 一次性创建多层目录
    - `-m` 指定权限
  - 用法
  - `mkdir -p project/src`  
`mkdir -m 700 secure_dir`
- 

#### 5. `rm` —— 删除文件/目录

- 常用选项
  - `-r` 删除目录及子目录

- `-f` 强制删除
- `-i` 删除前确认
- 用法
- `rm file.txt`
- `rm -rf olddir`

## 6. `rmdir` —— 删除空目录

- 常用选项
- `-p` 递归删除父目录（前提：都为空）
- 用法
- `rmdir emptydir`
- `rmdir -p top/second/third`

## 7. `touch` —— 修改时间戳 / 新建空文件

- 常用选项
- `-a` 修改 atime
- `-m` 修改 mtime
- `-t` 指定时间
- `-r` 参考文件
- 用法
- `touch newfile`
- `touch -t 202501011230.00 myfile`

## 8. `file` —— 判断文件类型

- 用法
- `file /etc/passwd`
- `file /bin/ls`
- 特点
- 不依赖扩展名
- 能识别文本/二进制/压缩/脚本等

### 记忆口诀

- 看 → `ls`
- 复制 → `cp`（目录要加 `-r`，备份常用 `-a`）
- 移动/改名 → `mv`
- 建目录 → `mkdir -p`
- 删文件/目录 → `rm -rf`（⚠危险）、空目录 → `rmdir`
- 改时间/建空文件 → `touch`
- 查类型 → `file`

## 2.1.2 メタキャラクターの利用（元字符の使用）

这一节讲的其实就是 **shell 里的通配符（wildcards）**，用来批量匹配文件名，很像 Windows 里的 `*.txt` 这种写法。

### ◆ 什么是元字符（メタキャラクター）

- 元字符就是一些特殊符号，代表**文件名的模式**。
- 作用：一次性处理多个文件，不用一个个输入。

- 注意：元字符展开（扩展）是 **由 shell 完成的**，不是命令本身在理解 `*` 或 `?`。

## ◆ 常见元字符（书上表 2-8）

1. `*`
  - 匹配 **0 个或更多任意字符**。
  - 例：`a*` → 能匹配 `a.txt`, `aaa`, `abc`
  - ⚠ 注意：能匹配 0 个字符，所以 `*` 也可能表示“啥都没有”。
2. `?`
  - 匹配 **任意 1 个字符**。
  - 例：`a?` → 匹配 `aa`, `ab`, `a1`, 但不匹配 `a` 或 `abc`。
3. `[]`
  - 匹配方括号里的 **任意一个字符**。
  - 例：`[abc].txt` → 匹配 `a.txt`, `b.txt`, `c.txt`
  - 还能用范围：`[a-z]` 表示任意小写字母；`[0-9]` 表示数字。
4. `{}`
  - 匹配 **逗号分隔的字符串集合**。
  - 例：`{d,e,f}.txt` → 会展开成 `d.txt e.txt f.txt`。
  - ⚠ 不止能匹配，还能**批量生成**文件名。

## ◆ 教材里的例子

假设目录里有文件：`a.txt aaa b.txt bbb c.txt ccc`

1. 匹配所有 `.txt` 结尾的文件：
2. `ls *.txt`
3. 📁 输出：`a.txt b.txt c.txt`
4. 匹配所有以 `a` 开头的文件：
5. `ls a*`
6. 📁 输出：`a.txt aaa`
7. 匹配以 `a` 或 `b` 开头，并且以 `.txt` 结尾的文件：
8. `ls [ab].txt`
9. 📁 输出：`a.txt b.txt`
10. 一次性生成多个文件：
11. `touch {d,e,f}.txt`
12. 📁 会生成：`d.txt e.txt f.txt`

## ◆ 重点提示（ここが重要）

1. 元字符展开是在 **shell 层面**完成的，不是命令本身。
  - 比如 `ls *.txt` → 实际执行的命令是 `ls a.txt b.txt c.txt`。
2. 如果你想把 `*` 当作普通字符用，而不是通配符，可以用 **反斜杠转义**：
3. `ls \*`
4. 📁 这样 `*` 就不会被当成“所有文件”的意思，而就是一个普通的星号。

## ✅ 总结记忆法：

- `*`：随便多少
- `?`：一个
- `[]`：选一个

- `{ }`：列一串

 元字符口诀表

| 符号  | 口诀         | 示例                | 说明                         |
|-----|------------|-------------------|----------------------------|
| *   | 星号多多，啥都能拖  | ls *.txt          | 匹配任意长度的字符串（0 个也行）          |
| ?   | 问号一句，只要一个  | ls a?             | 匹配任意 1 个字符                 |
| [ ] | 方框挑一个，范围都行 | ls [ab].txt       | 匹配中括号内的任意 1 个字符，支持范围 [a-z] |
| { } | 花括列队，批量全对  | touch {d,e,f}.txt | 批量展开成多个字符串                 |

  快速记忆

- \* → 想象成一张“万能卡”，啥都能代替。
- ? → 就像一道填空题，只能填 **一个** 字。
- [ ] → 就像一个“选一个”的菜单。
- { } → 就像把东西列出来，全部执行。

2.1.3 ファイルの検索

这个部分主要讲的是 Linux 下如何使用 `find` 命令来搜索文件和目录。📁

◆ `find` 命令的基本作用

- `find` 是 **Linux 中最强大的搜索命令之一**。
- 它能在指定的目录下递归地搜索文件或目录。
- 搜索条件不仅可以是文件名，还可以是：
  - **访问权限** (permission)
  - **文件大小** (size)
  - **最后访问/修改时间** (atime, mtime)
  - **所属用户** (user)
  - **文件类型** (type: 文件/目录/符号链接)
- 更厉害的是：匹配到的文件还能执行指定的命令，比如删除、复制。

◆ 基本语法

`find [搜索目录] [搜索条件]`

🔥 如果省略搜索目录，就从 **当前目录** 开始搜索。

◆ 示例讲解

1. 按文件名搜索

2. `find /root -name "*.txt"`

3. 📁 在 `/root` 下找所有以 `.txt` 结尾的文件。

⚠️ 但普通用户没有权限访问 `/root`，所以会报错：

`find: /root: Permission denied`

2. 按文件名后缀 `.md` 搜索

3. `find /usr/share/doc -name "*.md"`

4. 📁 在 `/usr/share/doc` 目录下，找所有 Markdown 文件。

🔥 注意：`"*.md"` 要加引号，避免 Shell 提前展开 `*`。



3. 找最近 1 天内修改过的文件

4. `find /home/linucuser -type f -mtime -1`
- `-mtime -1` 表示修改时间在 1 天内。
  - `-type f` 表示只找文件。

4. 找有 SUID 权限的文件

5. `find /usr/bin -type f -perm -u+s`
- `-perm -u+s`：表示有 SUID 位的文件（特殊权限）。

5. 找某个用户的文件

6. `find /tmp -user linucuser`
7. 📁 搜索 `/tmp` 下，所有 所属用户是 **linucuser** 的文件或目录。

6. 查找并删除 30 天没访问过的文件

7. `find . -atime +30 -type f -exec rm {} \;`
- `-atime +30`：30 天前访问过
  - `-exec rm {} \;`：对找到的文件执行 `rm` 删除操作
  - `{}` 会被替换为匹配到的文件名。

💎 小总结（记忆点）

1. **find = 搜索+动作**：不仅能查，还能对结果执行命令。
2. 常见条件：
- `-name "*.txt"`：按文件名
  - `-mtime -1`：修改时间
  - `-user`：按用户
  - `-perm`：按权限
  - `-type f/d`：按类型
3. 常见动作：
- `-print`（默认，显示结果）
  - `-exec ... {}`（执行命令）

📖 find 命令常见搜索条件（表2-9 整理）

| 选项 (日语原文)                    | 中文解释                        | 示例                                                             |
|------------------------------|-----------------------------|----------------------------------------------------------------|
| <code>-name</code> ファイル名     | 按文件名搜索                      | <code>find . -name "*.txt"</code> (查找所有 .txt 文件)               |
| <code>-atime</code> 日時       | 按最后访问时间搜索                   | <code>find /var/log -atime -7</code> (7天内访问过的文件)               |
| <code>-mtime</code> 日時       | 按最后修改时间搜索                   | <code>find . -mtime +30</code> (30天前修改的文件)                     |
| <code>-perm</code> アクセス権     | 按权限搜索                       | <code>find /usr/bin -perm -u+s</code> (有 SUID 权限的文件)           |
| <code>-size</code> サイズ       | 按文件大小搜索 (以块为单位)             | <code>find . -size +10M</code> (大于 10MB 的文件)                   |
| <code>-type</code> ファイルの種類   | 按文件类型搜索: f=文件, d=目录, l=符号链接 | <code>find . -type d</code> (查找目录)                             |
| <code>-user</code> ユーザー名     | 按文件所有者搜索                    | <code>find /home -user linucuser</code> (某用户的文件)               |
| <code>-print</code>          | 输出匹配结果 (默认可省略)              | <code>find . -name "*.sh" -print</code>                        |
| <code>-ls</code>             | 以 ls -l 形式显示详细信息            | <code>find . -type f -ls</code>                                |
| <code>-exec</code> コマンド {} ; | 对结果执行命令                     | <code>find . -name "*.log" -exec rm {} \;</code> (删除所有 log 文件) |
| <code>-ok</code> コマンド {} ;   | 和 -exec 类似，但会逐一确认           | <code>find . -name "*.log" -ok rm {} \;</code>                 |

  记忆方法

- 找名字 → `-name`

- 时间类 → `-atime` (访问时间)、`-mtime` (修改时间)
- 权限类 → `-perm`
- 大小类 → `-size`
- 类型类 → `-type`
- 用户类 → `-user`
- 操作类 → `-print` (显示)、`-exec` (执行)、`-ok` (确认执行)

## 📁 find 命令场景式记忆表

### 1. 想找某种名字的文件

👉 用 `-name`

```
find /usr/share/doc -name "*.md"
```

- 查找 `/usr/share/doc` 下所有以 `.md` 结尾的文件。  
📌 记忆: 名字 **name** → **-name**

### 2. 想找昨天 (1日内) 修改过的文件

👉 用 `-mtime -1`

```
find /home/linucuser -type f -mtime -1
```

- 查找 `/home/linucuser` 里 **1天以内修改**的文件。  
📌 记忆: **mtime = modification time** (修改时间)

### 3. 想找有特殊权限 (SUID) 的程序

👉 用 `-perm -u+s`

```
find /usr/bin -type f -perm -u+s
```

- 查找 `/usr/bin` 下设置了 **SUID 权限**的文件。  
📌 记忆: **perm = permission** (权限)

### 4. 想找某个用户的文件

👉 用 `-user`

```
find /tmp -user linucuser
```

- 查找 `/tmp` 下属于 `linucuser` 的文件。  
📌 记忆: **user** → **所有者**

### 5. 想找很久没用的文件并删除

👉 用 `-atime` + `-exec`

```
find . -atime +30 -type f -exec rm {} \;
```

- 查找当前目录下 **30天没访问过**的文件并删除。  
📌 记忆:
- **atime = access time** (访问时间)
- **exec = execute** (执行命令)
- `{}` → 每个找到的文件名
- `\;` → 结束符

🌟 ✅ 总结口诀:

- 按名字找 → **-name**
- 按时间找 → **-atime / -mtime**
- 按权限找 → **-perm**
- 按用户找 → **-user**

- 找完要操作 → `-exec / -ok`

## 2.1.4 ファイルの圧縮・解凍（文件的压缩与解冻）

这一节主要讲 Linux 里如何把文件“压缩（节省空间、方便传输）”以及如何“解压缩（恢复原文件）”。教材里依次介绍了 **gzip / bzip2 / xz** 三种压缩工具。下面我帮你分解讲解。

### 1. 为什么要压缩？

- 大文件如果直接保存或传输，占用空间太大、速度慢。
- 压缩后文件会更小（比如从 100MB 变成 30MB）。
- Linux 里常用的压缩命令就是：
  - **gzip** → 最常见
  - **bzip2** → 压缩率更高（比 gzip 更小），但速度慢
  - **xz** → 压缩率最高，但最慢

压缩后的文件扩展名：

- gzip → `.gz`
- bzip2 → `.bz2`
- xz → `.xz`

### 2. gzip 命令

#### 基本用法

```
gzip datafile
```

- 把 `datafile` 压缩为 `datafile.gz`，原始文件会被删除。

#### 常见选项

- **-d**：解压缩
- **-c**：输出到标准输出（可以重定向保存）
- **-k**：保留原文件，不删除
- **-r**：递归压缩目录里的所有文件

例子：

```
gzip -r sampledir # 压缩目录下所有文件
gzip -c datafile > datafile.gz # 保留原文件，同时生成压缩包
```

### 3. bzip2 命令

和 gzip 用法几乎一样，只是扩展名不同。

```
bzip2 datafile
```

- 生成 `datafile.bz2`，原文件删除。

解压缩：

```
bunzip2 datafile.bz2
```

选项：

- `-d` → 解压
- `-c` → 输出到标准输出
- `-k` → 保留原文件

### 4. xz 命令

压缩率最高的工具，用法也相同：

```
xz datafile
```

- 生成 `datafile.xz`。

解压缩：

```
xz -d datafile.xz
```

或者用：

```
unxz datafile.xz
```

选项：

- `-d` → 解压
  - `-k` → 保留原文件
  - `-l` → 查看压缩文件内容列表
- 

## 5. 总结记忆法

- 三个命令都是：命令 文件名 → 压缩
  - 三个解压命令：
    - `gzip` → `gunzip` 或 `gzip -d`
    - `bzip2` → `bunzip2` 或 `bzip2 -d`
    - `xz` → `unxz` 或 `xz -d`
  - 扩展名：
    - `.gz` → `gzip`
    - `.bz2` → `bzip2`
    - `.xz` → `xz`
- 

### 教材例子 1

```
$ gzip datafile
```

👉 把 `datafile` 压缩，生成 `datafile.gz`，原来的 `datafile` 会被删除。  
就像你把一个 Word 文档压缩成 ZIP 包，原文件被“打包走了”。

---

### 教材例子 2

```
$ gzip -r sampledir
```

👉 `-r` 表示递归，把 `sampledir` 目录里的所有文件逐个压缩。  
注意：它不会把目录本身打成一个包，只是目录里的文件一个个变成 `.gz`。  
(如果要打成一个整体压缩包，需要用 `tar`，我们后面学 2.1.5。)

---

### 教材例子 3

```
$ gzip -c datafile > datafile.gz
```

👉 `-c` 表示把压缩结果输出到标准输出（屏幕）。  
后面用 `>` 重定向保存成 `datafile.gz`。  
这样就能 **保留原始 datafile 文件**，同时额外得到一个压缩文件。  
相当于“复制一份压缩版本”，而不是“替换原文件”。

---

### 教材例子 4

```
$ bzip2 datafile
```

👉 和 `gzip` 一样，只是压缩率更高，结果是 `datafile.bz2`，原文件会删除。

---

### 教材例子 5

```
$ bunzip2 datafile.bz2
```

👉 解压缩 `.bz2` 文件，恢复出原来的 `datafile`。

教材例子 6

```
$ xz datafile
```

👉 压缩成 `datafile.xz`，原文件删除。

教材例子 7

```
$ xz -d datafile.xz
```

👉 解压缩 `.xz` 文件，恢复出原来的 `datafile`。  
同样也可以写成：

```
$ unxz datafile.xz
```

🔑 这里要记住的对照口诀

- `gzip` → `.gz`，解压是 **gunzip** 或 `gzip -d`
- `bzip2` → `.bz2`，解压是 **bunzip2** 或 `bzip2 -d`
- `xz` → `.xz`，解压是 **unxz** 或 `xz -d`

口诀：

👉 “**g b x → gz bz2 xz, gun bun un**”  
(`gzip` 对应 `gunzip`, `bzip2` 对应 `bunzip2`, `xz` 对应 `unxz`)

📊 Linux 压缩命令对比表

| 压缩命令  | 压缩后的扩展名 | 解压命令               | 特点                 |
|-------|---------|--------------------|--------------------|
| gzip  | .gz     | gunzip 或 gzip -d   | 压缩速度快，常用，压缩率一般     |
| bzip2 | .bz2    | bunzip2 或 bzip2 -d | 压缩率高于 gzip，但压缩速度更慢 |
| xz    | .xz     | unxz 或 xz -d       | 压缩率最高，但最慢，适合节省磁盘空间 |

2.1.5 アーカイブの作成・展開（tarコマンド）

◆ 什么是アーカイブ？

- **アーカイブ (Archive)**：把多个文件或目录“打包”成一个文件，方便保存或传输。
- 只是“打包”，不一定压缩。比如 Windows 的 `zip` 是打包 + 压缩，而 `tar` 默认只打包，不压缩。

◆ tar 命令的基本形式

```
tar [オプション] ファイル名あるいはディレクトリ名
```

常见选项（表2-13）：

| 选项       | 说明                    |
|----------|-----------------------|
| -c       | create, 创建归档文件        |
| -x       | extract, 从归档里解压文件     |
| -t       | list, 查看归档里有哪些文件      |
| -f       | file, 指定归档文件名（必需！）    |
| -z       | 使用 gzip 压缩/解压         |
| -j       | 使用 bzip2 压缩/解压        |
| -J       | 使用 xz 压缩/解压           |
| -v       | verbose, 显示详细处理信息     |
| -u       | update, 只追加比原有文件更新的文件 |
| -r       | append, 强行往归档里追加文件    |
| -N       | 指定日期后更新的文件才会包含        |
| --delete | 从归档里删除文件              |

⚠ 记忆窍门：

- **c x t f** = 创建、解压、查看、文件名 → 基本操作四件套
- 再配合 **z j J** = 压缩方式 (gzip/bzip2/xz)

## ◆ 实际例子

### 1. 创建归档

```
tar cvf /tmp/home.tar /home/linucuser
```

- **c** : 创建
- **v** : 显示详细
- **f** : 指定归档名

👉 把 `/home/linucuser` 打包成 `/tmp/home.tar`

### 2. 解压归档

```
tar xvf software.tar.gz
```

- **x** : 解压
- **v** : 显示文件列表
- **f** : 指定归档文件
- **z** : 表示这是 gzip 压缩的

👉 把 `software.tar.gz` 解压到当前目录

### 3. 查看归档内容

```
tar tzf software.tar.gz
```

- **t** : 只列出归档文件内容, 不解压
  - **z** : gzip
- 👉 输出: LICENSE, README, configure, ...

### 4. 解压指定文件

```
tar xvf /tmp/home.tar home/linucuser/.bash_profile
```

👉 只把 `.bash_profile` 取出来, 不会解压整个归档。

## ◆ 要点总结

1. **tar 主要是打包**, 是否压缩要加 **z j J**。
2. 基本三件套:
  - `tar cvf` → 创建
  - `tar xvf` → 解压
  - `tar tvf` → 查看
3. 记忆口诀:
  - **c (create) x (extract) t (table/list) f (file)**
  - 压缩方式: **z (gzip) j (bzip2) J (xz)**

## tar 归档与压缩的完整流程

### 1. 创建归档

```
tar cvf archive.tar files
```

### 2. 压缩归档

- gzip: `tar cvzf archive.tar.gz files`
- bzip2: `tar cjvf archive.tar.bz2 files`
- xz: `tar cJvf archive.tar.xz files`

### 3. 查看内容

```
tar tvf archive.tar
```

### 4. 解压归档

```
tar xvf archive.tar
```

### 5. 解压压缩归档

- `tar xvzf archive.tar.gz`
- `tar xvjf archive.tar.bz2`
- `tar xvJf archive.tar.xz`

这样就能清楚记住流程：先 **tar 打包** → 再用不同算法压缩 → **查看内容** → **解压**。

💡 📌 口诀：打包看解压，zjb 算法压

- **打包** → `tar cvf`
- **看内容** → `tar tvf`
- **解压** → `tar xvf`
- **压缩算法记字母**：
  - `z` → gzip
  - `j` → bzip2
  - `J` → xz

🌟 记忆法：

只要看到 `z j J` 就想：**gzip、bzip2、xz**，  
组合在 `tar` 里用：`tar c/z/j/Jvf`、`tar x/z/j/Jvf`。

## 2.1.6 dd コマンド

### 📌 dd 命令

#### 作用

- `cp`：只能复制文件（普通用法）。
- `dd`：可以复制 **文件**，也可以复制 **磁盘、分区**，甚至做系统镜像。  
所以 `dd` 很危险，常被戏称为 **disk destroyer（磁盘毁灭者）**。

#### 基本格式

```
dd if=输入文件 of=输出文件 bs=块大小 count=次数
```

#### 常用参数

- `if=` → **input file** 输入文件（默认是标准输入）
- `of=` → **output file** 输出文件（默认是标准输出）
- `bs=` → **block size** 每次复制的块大小（比如 1M）
- `count=` → **复制多少个块**

#### 示例

##### 487. 文件复制

```
dd if=oldfile of=newfile
```

把 `oldfile` 内容原封不动复制到 `newfile`。

##### 2. 磁盘克隆

```
dd if=/dev/sdb of=/dev/sdc
```

把 `/dev/sdb`（U盘）里的内容，整个拷贝到 `/dev/sdc`（另一块硬盘）。

👉 分区表和数据都会被一并复制。

##### 3. 只复制一部分

```
dd if=source of=dest bs=1M count=100
```

一次读取 1MB，总共复制 100MB。

🔑 考点总结 (记忆法)

- `if=` → input file (输入源)
- `of=` → output file (输出目的地)
- `bs=` → block size (一次复制多大块)
- `count=` → 总共复制多少块

👉 一句话记忆：  
**dd = 危险版 cp，可以直接操作磁盘，参数记住 if/of/bs/count。**

📊 cp vs dd 对比表

| 特点        | cp          | dd                                      |
|-----------|-------------|-----------------------------------------|
| 用途        | 复制普通文件/目录   | 复制文件、磁盘、分区，制作镜像                         |
| 常用格式      | cp 源文件 目标文件 | dd if=输入文件 of=输出文件 bs=块大小 count=次数      |
| 默认输入/输出   | 文件路径        | 默认 if=标准输入，of=标准输出                      |
| 是否能复制设备文件 | ❌ 不能        | ✅ 可以 (如 /dev/sda，整个磁盘内容)                |
| 常见参数      | -r (递归复制目录) | if (输入文件)、of (输出文件)、bs (块大小)、count (次数) |
| 使用风险      | 低 (最多覆盖文件)  | 高 (可能覆盖整个磁盘 → “磁盘毁灭者”)                  |
| 适合场景      | 日常文件拷贝      | 系统备份、磁盘克隆、提取镜像                          |

🌳 考试记忆口诀

- **cp = 文件复制** (Copy)
- **dd = 数据复制** (Disk/Device Destroyer 😬)
- dd 关键参数: **if / of / bs / count**

2.2 ファイルの所有者とパーミッション

2.2.1 所有者 (Owner)

◆ 文件的所有者 (Owner)

在 Linux 中，每个文件和目录都会关联 **所有者 (owner)** 和 **所属组 (group)**。

1. 所有者 (Owner)

- 通常是创建该文件的用户。
- 比如用户 `linucuser` 创建了文件 → 这个文件的所有者就是 `linucuser`。

2. 所属组 (Group)

- 每个用户都会属于至少一个组 (Primary Group)。
- 文件的所属组会默认设为创建该文件的用户所属的 **Primary Group**。
- 例如：用户 `linucuser` 属于组 `linux`，那么创建的文件 → 所属组就是 `linux`。

🔑 示例 (教材中的命令)

```
$ ls -l testfile
-rw-rw-r-- 1 linucuser linux 34 Jan 12 20:36 testfile
```

分解解释：

- **-rw-rw-r--** → 文件权限 (r=读, w=写, x=执行)
- **1** → 硬链接数
- **linucuser** → 文件所有者 (Owner)



- **linux** → 文件所属组 (Group)
- **34** → 文件大小 (字节数)
- **Jan 12 20:36** → 最后修改时间
- **testfile** → 文件名

### ⚠ 注意

- 多数情况下，用户所属组名和用户名相同。  
例如：用户 `linucuser`，它的默认组 (Primary Group) 也是 `linucuser`。
- 但在教材里，例子用了组名 `linux`。这说明 `linucuser` 属于组 `linux`。

## 2.2.2 アクセス権 (访问权限)

### 1. 什么是アクセス権 (权限)

- **アクセス権 (Permission)** : 决定“谁 (Which user) ”可以对“文件或目录”做“什么操作”。
- 可以针对三类用户设置：
  - fi. 所有者 (Owner)
  - fj. 所属グループ (Group)
  - fk. その他のユーザー (Others, 其余用户)
- 三种基本权限：
  - ip. **r (読み取り可能, Read)** → 可以读取文件内容 / 查看目录中的文件列表
  - iq. **w (書き込み可能, Write)** → 可以修改文件内容、删除文件 / 在目录中创建或删除文件
  - ir. **x (実行可能, Execute)** → 可以执行文件 (如果是程序、脚本) / 进入目录

### 2. 文件与目录的权限意义

教材里分开解释了：

#### 📁 文件的权限

- **r** → 可以读取文件内容
- **w** → 可以修改文件内容，或删除
- **x** → 可以作为程序执行

#### 📁 目录的权限

- **r** → 可以查看目录下文件列表
- **w** → 可以在目录中新建文件或删除文件
- **x** → 可以进入该目录并访问其中文件

### 3. 权限的表示方法

在 Linux 中用 `ls -l` 可以看到权限，例如：

```
$ ls -l
-rwxr-xr-- 1 linucuser linucuser 33 Jan 12 20:39 sample.sh
```

结果分解：

- **-** : 文件类型 (`-` 表示普通文件, `d` 表示目录, `l` 表示符号链接)
- **rwX** : 所有者权限 (读/写/执行)
- **r-X** : 组权限 (读/执行)
- **r--** : 其他用户权限 (只读)

🔗 所以 `-rwxr-xr--` 表示：

- Owner → 读、写、执行
- Group → 读、执行

- Others → 只读

#### 4. 数字表示法（八进制）

Linux 还常用数字来表示权限：

- $r = 4, w = 2, x = 1$
- 把权限相加得到数字：

例如：

- $rw\bar{x} \rightarrow 4+2+1 = 7$
- $rw- \rightarrow 4+2 = 6$
- $r-- \rightarrow 4$

于是：

- `-rwxr-xr--` → **754**
  - Owner: 7 (rwx)
  - Group: 5 (r-x)
  - Others: 4 (r--)

#### 5. 修改权限：chmod

教材举了两个例子：

##### 1. 符号法：

```
chmod go+w testfile
```

👉 给 group 和 others 增加写权限

##### 2. 数字法：

```
chmod 644 testfile
```

👉 修改成所有者可读写，组和其他用户只读（ $rw-r--r--$ ）

#### 📌 记忆技巧：

- 文件权限重点记：r=读内容，w=改内容，x=执行
- 目录权限重点记：r=能看目录，w=能改目录内容（新建/删除），x=能进入目录
- 数字记忆口诀： $r=4, w=2, x=1 \rightarrow$  组合成7/6/5/4

##### • r (read)

- 文件：能看里面的内容。
- 目录：能看到目录里的清单。

##### • w (write)

- 文件：能改、能删。
- 目录：能新增、能删掉文件。

##### • x (execute)

- 文件：能运行。
- 目录：能进入（cd）并访问里面的文件。

👉 可以把目录想象成「一个文件柜」：

- **r** 是能翻看目录里的清单；
- **w** 是能往柜子里加/删文件夹；
- **x** 是能打开柜子、进去里面操作。

2.2.3 SUID, SGID

SUID (Set User ID)

- 定义：当文件带有 **SUID** 权限时，用户执行该文件时，会以**文件所有者**的权限来运行，而不是执行用户本人的权限。
- 例子：

```
/usr/bin/passwd
```

 命令拥有 SUID。
  - 普通用户执行 `passwd` 时，实际上是以 root 的权限在修改 `/etc/shadow` 文件。
  - 因为 `/etc/shadow` 文件只有 root 才能写入。
- 表示方式：
  - 权限中表现为：`rws`（如：`-rwsr-xr-x`）。
  - 数字表示：在普通权限前加上 **4000**。
    - 例如：`4755` → 表示拥有 SUID 的可执行文件。

设置方法：

```
chmod u+s samplefile
```

SGID (Set Group ID)

- 定义：当目录带有 **SGID** 时，该目录中新建的文件和子目录，会自动继承**目录所属的组**，而不是新建者的主组。
- 作用：适合多个用户需要共享一个目录的场景。
  - 例如：在一个团队共享目录中，新建的文件自动归属团队的组。
- 表示方式：
  - 权限中表现为：`rws`（组的执行位变为 s）。
  - 数字表示：在普通权限前加上 **2000**。
    - 例如：`2755` → 表示拥有 SGID 的目录。

设置方法：

```
chmod g+s samplefile
```

重点总结

- SUID**：执行时临时提升为**文件所有者的权限**（常用于程序，如 `passwd`）。
- SGID**：新建文件继承**目录的组**（常用于共享目录）。

SUID 和 SGID 对比

| 特性   | SUID (Set User ID)                                      | SGID (Set Group ID)                      |
|------|---------------------------------------------------------|------------------------------------------|
| 作用对象 | 文件 (常是执行程序)                                             | 目录 (也可用于文件)                              |
| 执行效果 | 执行文件时，临时以 <b>文件所有者</b> 的权限运行                            | 在目录中新建文件时，自动继承该目录的 <b>所属组</b>            |
| 权限表示 | 在 <b>用户 (owner)</b> 的执行位 (x) 位置显示 s → rws               | 在 <b>组 (group)</b> 的执行位 (x) 位置显示 s → rws |
| 数值表示 | 在权限前加上 <b>4000</b> → 4755                               | 在权限前加上 <b>2000</b> → 2755                |
| 常见用途 | <code>passwd</code> 程序：普通用户能修改 <code>/etc/shadow</code> | 团队共享目录：新文件自动属于团队组                        |
| 命令设置 | <code>chmod u+s file</code>                             | <code>chmod g+s dir</code>               |

一句话记忆法：

- SUID** → **提权执行**（执行别人文件时，借用“主人”的权限）。
- SGID** → **组内继承**（新文件都跟“家族”走）。

2.2.4 スティッキービット (Sticky Bit)

1. 什么是 Sticky Bit?

Sticky Bit 是 Linux 权限的一种特殊标志，主要用于**目录**。

- 如果目录设置了 Sticky Bit，那么即使用户对该目录有写权限，也**不能删除或修改其他用户创建的文件**。
- 只能删除或修改**自己拥有的文件**。

👉 典型的例子就是 `/tmp` 目录。

- `/tmp` 是一个所有人都能写入的临时目录。
- 如果没有 Sticky Bit，任何用户都可以删除别人的文件，系统就会变得不安全。
- 有了 Sticky Bit，用户就只能删除自己文件，保证共享目录的安全。

## 2. 显示方式

在 `ls -ld` 输出结果中，如果设置了 Sticky Bit：

- 目录权限最后一个 **执行权限位 x** 会显示为 **t** 或 **T**。
  - **t**：该目录对“其他用户”有执行权限，并启用了 Sticky Bit。
  - **T**：该目录对“其他用户”没有执行权限，但启用了 Sticky Bit。

🔥 例子：

```
$ ls -ld /tmp
drwxrwxrwt 8 root root 211 Jan 13 22:18 /tmp
```

最后一位是 **t**，表示 `/tmp` 目录设置了 Sticky Bit。

## 3. 设置 Sticky Bit

- 使用 **chmod** 命令：

(1) 符号方式：

```
chmod o+t 目录名
```

例如：

```
chmod o+t testdir
```

(2) 数字方式：

在权限数字前加 **1xxx**（1000 表示 Sticky Bit）。

```
chmod 1777 目录名
```

**1777** 表示：

- 所有人可读写执行（777）
- 同时启用了 Sticky Bit（+1000）

## 4. 总结

- Sticky Bit 只对**目录**有意义。
- 常见的应用场景：**`/tmp` 目录**。
- 作用：保证多人共享目录时，用户只能删除自己文件，不能删除别人的。

## 2.2.5 デフォルトのアクセス権（默认访问权限）

### 1. 什么是默认访问权限？

当我们在 Linux 中新建一个文件或目录时，系统会自动给它设置一组初始权限。

这个默认权限并不是固定值，而是由 **umask（用户掩码值）** 来决定的。

- **文件默认权限**：`666`（即所有用户都有读写权限，但没有执行权限）。
- **目录默认权限**：`777`（即所有用户都有读、写、执行权限）。

然后，系统会用 **umask 值来减掉相应的权限**，得到最终的实际权限。

### 2. umask 的概念

**umask** 就是 **User Mask**，表示要“去掉”的权限。

- 它的本质是 **对默认权限取反操作**。
- 常用 3 位（有时显示 4 位，最前一位是特殊权限位）。

### 3. 示例

(1) 查看当前 umask 值：

```
$ umask
0002
```

实际有效的是后三位：002。

- 文件默认权限 =  $666 - 002 = 664 \rightarrow rw-rw-r--$
- 目录默认权限 =  $777 - 002 = 775 \rightarrow rwxrwxr-x$

(2) 修改 umask 值：

```
$ umask 027
```

再新建文件和目录：

```
$ touch umasktest
$ mkdir umasktestdir
$ ls -ld umask*
-rw-r----- 1 linucuser linucuser ... umasktest
drwxr-x--- 2 linucuser linucuser ... umasktestdir
```

结果：

- 文件权限：640
- 目录权限：750

因为 027 表示：

- 去掉“组用户”的写权限
- 去掉“其他用户”的所有权限

#### 4. 总结

- umask = 要去掉的权限**
- 文件默认基准：666
- 目录默认基准：777
- 实际权限 = 基准 - umask

#### ▶ 2.2.6 所有者ノ管理

#### ▶ 2.2.7 グループノ管理

## 2.3 ハードリンクとシンボリックリンク

Linux 里，文件和目录可以通过不同的名字来访问同一个数据，这就是[链接](#)。分为两种：

1. ハードリンク (Hard Link, [硬链接](#))
2. シンボリックリンク (Symbolic Link, [符号链接/软链接](#))

### 2.3.1 ハードリンク（硬链接）

原理

- 每个文件在磁盘上都有一个唯一的 **inode 番号 (inode 号)**。
- 硬链接**就是给同一个 inode 再取一个新的文件名。
- 多个硬链接文件其实指向的是**同一块物理数据**。
- 因此，无论你用哪个名字访问，看到的内容都是一致的。

特点

- 文件的实质数据只有一份，所有硬链接共享同一个 inode。
- 删除一个硬链接不会删除实际数据**，只有当所有硬链接都被删除后，文件数据才会真正消失。
- 限制**：
  - 不能跨文件系统（不同的磁盘分区不行）。

- 不能对目录创建硬链接（避免目录循环）。

例子

```
$ ls -li file*
8409155 -rw-r--r-- 3 linucuser linucuser 662 Jan 16 23:45 file.org
8409155 -rw-r--r-- 3 linucuser linucuser 662 Jan 16 23:45 file2
8409155 -rw-r--r-- 3 linucuser linucuser 662 Jan 16 23:45 file3
```

👉 inode号 (8409155) 相同，说明它们是硬链接，链接数为 3。

2.3.2 シンボリックリンク（符号链接）

原理

- 符号链接其实是一个指向目标文件路径的特殊文件。
- 类似于 **Windows 的快捷方式** 或 macOS 的 **エイリアス**。
- 自身有独立的 inode，只是内容是「目标文件路径」。

特点

- 可以跨文件系统。
- 可以对目录创建符号链接。
- 如果目标文件被删除，符号链接会变成「断链 (Broken Link)」。

例子

```
$ ln -s fileA fileB
```

👉 fileB 就是 fileA 的符号链接。删除 fileA 后，fileB 会失效。

2.3.3 リンクの作成

使用 `ln` 命令：

- 硬链接**（默认）：  
`ln 源文件 链接名`
- 符号链接**：  
`ln -s 源文件 链接名`

2.3.4 リンクのコピー

符号链接在复制时有两种情况：

- 默认 `cp` → 会复制目标文件的内容，链接会变成普通文件。
- 使用 `-d` → 会复制为新的符号链接。

```
$ cp linktest.link_sym linktest2 # 普通复制（内容复制）
$ cp -d linktest.link_sym linktest3 # 保留为符号链接
```

总结对比表

| 特性     | 硬链接 (Hard Link) | 符号链接 (Symbolic Link) |
|--------|-----------------|----------------------|
| inode  | 相同              | 不同                   |
| 删除目标文件 | 不受影响            | 链接失效                 |
| 跨文件系统  | 不行              | 可以                   |
| 指向目录   | 不行              | 可以                   |
| 本质     | 同一文件的别名         | 指向路径的快捷方式            |

👉 考试重点通常是：

- 区分 **硬链接 vs 符号链接**。
- `ln` 和 `ln -s` 的用法。

- 3. inode号和 link count 的意义。
- 4. 符号链接断链的现象。

🔗 链接对比速记表

| 项目     | ハードリンク (Hard Link) | シンボリックリンク (Symbolic Link) |
|--------|--------------------|---------------------------|
| inode  | 相同 (指向同一个 inode)   | 不同 (有独立 inode, 内容是路径)     |
| 删除原文件  | 不影响 → 其他硬链接还能访问    | 链接会失效 (断链, Broken Link)   |
| 跨文件系统  | ❌ 不可以              | ✅ 可以                      |
| 指向目录   | ❌ 不可以              | ✅ 可以                      |
| 链接数量显示 | ls -l 的第2列 Link数增加 | 不影响 Link数                 |
| 本质     | 同一文件的别名            | 快速方式/路径指针                 |
| 创建命令   | ln 源文件 链接名         | ln -s 源文件 链接名             |
| 常见考点陷阱 | inode 相同、删除不影响     | 删除目标文件后会出错                |

🗝️ 记忆口诀

- 硬 (Hard) = 实体共享 → 真正的文件数据被多个名字共享。
- 符 (Symbolic) = 符号指路 → 只是指向目标路径，目标没了就迷路。

2.4 文件の配置と検索

这一节的内容。我们分为两个部分：

- 1. 文件与目录的典型位置 (FHS标准)
- 2. 文件搜索相关命令

2.4.1 典型的文件和目录位置 (FHS)

FHS (Filesystem Hierarchy Standard) 规定了 Linux 系统中文件和目录的标准布局。主要目录的作用如下：

- **/bin**  
存放基本命令（普通用户和 root 都能使用），如 `ls`，`cp`，`mv`，`cat`，`chmod`。
- **/sbin**  
存放系统管理命令（通常 root 使用），如 `shutdown`，`fdisk`，`fsck`，`reboot`。
- **/etc**  
系统配置文件和脚本，如 `/etc/passwd`，`/etc/fstab`。
- **/dev**  
设备文件目录，把硬件设备抽象为文件，比如 `/dev/sda`，`/dev/null`。
- **/lib**  
系统运行需要的共享库和内核模块，支撑 `/bin` 和 `/sbin` 中命令。
- **/media**  
移动介质的挂载点（U盘、DVD 等）。
- **/mnt**  
临时挂载点，手动挂载时常用。
- **/opt**  
第三方软件安装位置（包管理机制之外的软件）。
- **/proc**  
虚拟文件系统，提供内核和进程信息，例如 `/proc/cpuinfo`。
- **/root**  
root 用户的主目录。
- **/boot**  
系统启动所需的内核和引导加载文件。
- **/home**  
普通用户的主目录（如 `/home/linuxuser`）。

- **/tmp**  
临时文件存放地，所有用户可读写。
  - **/var**  
可变数据（日志、缓存、队列等）。
    - `/var/cache` 缓存文件
    - `/var/lock` 锁文件
    - `/var/log` 日志文件
    - `/var/run` 运行时信息
    - `/var/spool` 打印任务、邮件队列
  - **/usr**  
大量用户程序和库。
    - `/usr/bin` 常用命令（普通用户使用）
    - `/usr/sbin` 系统管理命令（非必须紧急用）
    - `/usr/lib` 共享库
    - `/usr/local` 本地安装的程序和库
    - `/usr/share` 架构无关文件（如 man 文档）
    - `/usr/src` 源码（如内核源代码）
- 

## 2.4.2 文件的搜索

Linux 提供多个命令搜索文件，各有特点：

### 1. locate

- 基于预先建立的数据库（由 `updatedb` 生成）。
- 搜索速度快，但可能查不到刚创建或修改的文件。
- 用法示例：

```
$ locate "*.h"
```
- 查找以 `.h` 结尾的文件。

### 2. updatedb

- 重新生成 `locate` 使用的数据库。
- 可以用 `-e` 排除目录：

```
updatedb -e /tmp
```

### 3. which

- 搜索命令的绝对路径（只查 `$PATH` 路径下）。
- ```
$ which useradd
/usr/sbin/useradd
```

4. whereis

- 搜索命令的二进制文件、源码和 man 手册路径。
- 选项：
 - `-b` 只搜索二进制
 - `-m` 只搜索手册
 - `-s` 只搜索源码
- ```
$ whereis useradd
useradd: /usr/sbin/useradd /usr/share/man/man8/useradd.8.gz
```

### 5. type

- 判断一个命令属于哪种类型：
  - 外部命令（执行文件）
  - 内建命令（shell 内置）
  - alias（别名）



- shell 关键字
- ```
$ type cat
cat is /bin/cat
$ type echo
echo is a shell builtin
$ type ls
ls is aliased to `ls --color=auto`
$ type for
for is a shell keyword
```

✅ 总结：

- **FHS** 让不同 Linux 发行版有统一的目录结构。
- **locate + updatedb**：快速文件搜索。
- **which/whereis/type**：帮助你定位命令的来源、类型和路径。

📄 文件的配置と検索 速记表

💠 典型目录（FHS）

目录	作用
/bin	基本命令（所有用户可用）
/sbin	系统管理命令（root 专用）
/etc	系统配置文件（passwd, fstab 等）
/dev	设备文件（硬盘、终端等）
/lib	必要的共享库、内核模块
/media	可移动介质挂载点（U盘/DVD）
/mnt	临时挂载点
/opt	第三方软件安装位置
/proc	内核和进程信息（虚拟文件系统）
/root	root 用户的家目录
/boot	系统启动文件、内核
/home	普通用户家目录
/tmp	临时文件（所有用户可读写）
/var	变化频繁的数据（日志、缓存、队列）
└ /var/cache	缓存数据
└ /var/lock	程序锁文件
└ /var/log	系统日志
└ /var/run	运行时信息
└ /var/spool	打印、邮件队列
/usr	大部分用户程序和库
└ /usr/bin	普通命令（非紧急）
└ /usr/sbin	系统管理命令（非紧急）
└ /usr/lib	程序共享库
└ /usr/local	本地安装的程序
└ /usr/share	架构无关文件（man文档等）
└ /usr/src	源码（如内核源码）

💠 文件搜索命令

命令	说明	示例
locate	基于数据库快速搜索文件	locate "*.h"
updatedb	更新 locate 用的数据库	updatedb -e /tmp（排除 /tmp）
which	显示命令的绝对路径（\$PATH 内）	which useradd
whereis	搜索命令的二进制、源码、man 手册	whereis useradd
type	判断命令类型（外部、内建、alias、关键字）	type ls

- `/bin` 和 `/sbin` → 基本命令 vs 系统管理命令
- `/etc` → 系统配置
- `/dev` → 设备文件
- `/var/log` → 日志文件位置
- **locate** 比 **find** 快, 但依赖 `updatedb` 数据库
- **which** 只查 `$PATH` 内的命令
- **type** 能区分命令类型 (外部命令/内建/alias/关键字)